

Programmeren Gevorderd: Opdracht 'Zorgcentrum'

1. Opdracht 'Zorgcentrum'

In deze opdracht ga je een volledige applicatie ontwikkelen in meerdere lagen. Als examen ga je deze mondeling verdedigen.

2. Verwachte werkwijze

Deze **semesteropdracht** is geen opdracht die je pas op het einde van het semester uitwerkt. Integendeel: **de bedoeling is dat je project stap voor stap groeit** naarmate nieuwe concepten en technieken tijdens de lessen aan bod komen.

Tijdens de lessen werken we aan verschillende onderdelen van een meerlagige applicatie. Je wordt sterk aangemoedigd om de aangereikte leerstof onmiddellijk toe te passen binnen je eigen project. Op die manier evolueert je eindopdracht **parallel met het lesverloop** en bouw je geleidelijk een volledige oplossing uit.

Het is daarbij belangrijk dat je **regelmatig werkt** aan je project en je voortgang zichtbaar maakt via **Git**. Elke betekenisvolle uitbreiding of verbetering van de applicatie wordt vastgelegd in een afzonderlijke commit met een duidelijke commitboodschap. Hierdoor ontstaat een overzichtelijke historiek die aantoont hoe je project stelselmatig gegroeid is gedurende het semester.

Van studenten wordt verwacht dat zij:

- vanaf de start van het semester een werkende projectstructuur opzetten;
- de inhoud van de lessen systematisch verwerken in hun project;
- regelmatig committen naar hun Git-repository;
- duidelijk kunnen aantonen hoe hun applicatie evolueerde doorheen het semester;
- niet wachten tot de laatste weken om grote delen van de opdracht te realiseren;
- de principes van meerlagige softwareontwikkeling consequent toepassen.

De Git-historiek maakt deel uit van de evaluatie. Een project dat pas op het einde van het semester in één of enkele grote commits wordt toegevoegd, strookt niet met de beoogde werkwijze. Er wordt verwacht dat de git-repository een realistisch ontwikkeltraject weerspiegelt, waarbij de verschillende onderdelen van de applicatie geleidelijk werden opgebouwd, getest en verbeterd.

Het uiteindelijke doel is niet alleen een correct eindresultaat opleveren, maar ook aantonen dat je een professioneel ontwikkelproces kunt volgen, waarin analyse, implementatie, testen en versiebeheer op een gestructureerde manier worden toegepast.

3. Mondelinge verdediging

Tussentijds

- **Deadline oplevering in Orion: Einde week 8: 15 november 2026**
- Mondelinge verdediging op afspraak tijdens week 9 → ook als voorbereiding op de projectweek in week 10;

Eindverdediging

- Deadline oplevering in Orion: wordt nog meegeeld via Orion
- Mondelinge verdediging op afspraak tijdens de examenperiode.

Tijdens de mondelinge verdediging demonstreer je de werking van je applicaties en licht je de gemaakte ontwerp- en implementatiekeuzes toe. Je beantwoordt daarbij gerichte vragen van de lector over zowel de functionaliteit als de onderliggende code.

Tijdens deze verdediging ontvang je bovendien mondelinge feedback. Verwacht wordt dat je deze feedback analyseert en verwerkt in een volgende versie van de applicatie en/of tijdens een volgende verdediging.



Wie de opdracht niet indient voor de deadline wordt niet uitgenodigd voor de mondelinge verdediging. Het rooster voor mondelinge verdediging wordt kort na de deadline bekendgemaakt.



Dien gerust meerdere keren in op Orion, enkel de laatste versie wordt bekeken en verbeterd maar zo ben je wel bestand tegen calamiteiten op de dag van de deadline.

4. Hoe indienen

Bij het indienen wordt verwacht:

- Er is een Zorgcentrum opdracht in Orion
- Op te laden is een zip file met daarin een folder: **Zorgcentrum - Jouw naam en voornaam**.
 - daarin o.a. de volledige solution en dit zonder de subfolders **bin** en **obj**.
 - Een export van je schema's en data uit je database in een **.sql** script
 - Exporteer alles uit de database: **schema en data** (⇒ zoek zelf op hoe dit moet)
 - De export bestaat uit een tekst bestand met SQL queries om de database tabellen aan te maken enerzijds en om deze op te vullen met data anderzijds. Je kan deze automatisch laten genereren.

5. Context opdracht

In een zorgcentrum met meerdere **Afdelingen** komen **Artsen** en **Patiënten** samen.

Het is aan jou om o.a. een applicatie te bouwen voor het beheren van afspraken.

Uit meerdere gesprekken met het zorgcentrum konden we belangerijke eisen van de klanten afleiden:

- Er bestaat reeds data die hergebruikt moet worden. Deze data wordt aangeleverd in tekstbestanden;
- Het zorgcentrum praat over concepten als Patiënt, Arts, Afspraak etc. Aan deze concepten zijn voor hen een aantal relaties en regels verbonden;
- Het zorgcentrum verwacht unit-tests van zijn software omdat dit zorgt voor hogere betrouwbaarheid, minder fouten en lagere onderhoudskosten op de lange termijn. Schrijf unit testen voor de domeinregels.

5.1. Bestaande data hergebruiken

Het zorgcentrum stelt ons enkele tekstbestanden ter beschikking met data om van te vertrekken. Deze data kan fouten bevatten.

Doel

Je bouwt twee grafische applicaties:

1. Bouw een eerste applicatie die deze tekstbestanden inleest, domeinregels afdwingt en de data in een relationele databank plaatst (jij ontwerpt ook het ERD).
2. Daarna bouw je een WPF-gebruikersapp om met deze gegevens te kunnen werken.

5.1.1. Info bestaande gegevens

Beschikbare gegevens die ons in tekstbestanden werden doorgestuurd hebben volgende structuur:

Artsen.csv

```
RijksregisterNr;Naam;Voornaam;RIZIV;Specialisatie;Afdelingen
61.08.05-432.76;ABDUL-LATIF;KHALID;1-79010-52-100;{ANES};{ORTH}|{PAAN}|{URGE}
69.08.06-893.44;ACCOU;GERALDINE;1-48841-54-370;{OPHT};{PAAN}|{RADI}
96.05.02-114.34;ACHTERGAEL;WIM;1-76821-10-660;{ONCO};{ONCO}|{RADI}
73.12.28-670.10;ADAM;VALERIE;1-19942-47-620;{PNEU};{PAAN}|{RADI}
...
```

Artsen hebben één specialisatie en kunnen verbonden zijn aan meerdere afdelingen.

Patiënten.csv

```
RijksregisterNr;Naam;Voornaam;Afdelingen
63.01.02-359.38;Boddaert;Daisy;{NEUR}
15.01.23-042.07;De Somviele;Marie-Christine;{URGE}
48.03.26-472.68;Roels;Romuald;{GERC,ONCO,RADI}
56.01.10-336.62;Raeymaekers;Eva;{GASTG,URGE}
```

...

Patiënten kunnen ingeschreven zijn in meerdere afdelingen.

Afspraken.csv

```
AfspraakId;Type;DatumTijdStart;DuurMin;RijksregisterNrPatiënt;RijksregisterNrArts;Afde-
lingCode;Extra1;Extra2
1;CONSULTATIE;2026-04-01T09:15;20;60.11.20-620.20;38.12.17-278.94;GERC
1341;OPERATIE;2026-05-14T10:00;90;82.04.15-057.79;38.12.17-278.94;GERC;Nefrectomie;90
2434;CONTROLE;2026-06-18T09:00;15;82.04.15-057.79;38.12.17-278.94;GERC;ref=1341
...
```

Uitleg bij sommige kolommen in de afspraak gegevens:

- **Type** is een element van deze set {CONSULTATIE, OPERATIE, CONTROLE}.
- **Extra1:**
 - voor een OPERATIE is dit de beschrijving van de ingreep.
 - voor CONTROLE bevat dit `ref=<AfspraakId>`, een referentie naar de afspraak waarop wordt gecontroleerd.
- **Extra2:** Aantal minuten dat de patiënt op voorhand dient aanwezig te zijn voor een operatie. Minstens 30 minuten.

5.1.2. Robuustheid

Het zorgcentrum is zich ervan bewust dat de aangeleverde gegevens fouten kunnen bevatten. Daarom verwacht het dat tijdens de verwerking van de tekstbestanden alle gedetecteerde fouten systematisch worden geregistreerd en gerapporteerd.

De foutafhandeling moet minstens voorzien in het detecteren van problemen zoals duplicaten, onbekende codes, ongeldige sleutels, overlappende afspraken per arts, ongeldige duurwaarden, ongeldige referenties, ongeldige rijksregisternummers (RRN), ongeldige RIZIV-nummers en andere soortgelijke fouten.

Voor de vastgestelde problemen dienen één of meerdere foutbestanden te worden gegenereerd. Aangezien de gegevens via tekstbestanden worden aangeleverd, moet elk foutbestand minstens de volgende informatie bevatten:

- het lijnnummer waarop de fout werd vastgesteld;
- de volledige inhoud van de betreffende lijn;
- een beschrijving van één of meerdere fouten die in die lijn aanwezig zijn.

5.2. Relaties

Volgende relaties dien je te volgen:

- Een Consultatie **IS EEN** Afspraak;
- Een Operatie **IS EEN** Afspraak;
- Een Controle **IS EEN** Afspraak.

Merk ook op dat een Arts en een Patiënt allebei personen zijn. Maar een arts kan ook als patiënt ingeschreven worden in hetzelfde ziekenhuis. Maak in je programma duidelijk onderscheid tussen artsen en patiënten. Een persoon kan nooit tegelijkertijd aanzien worden als arts en als patient.

5.3. Domeinregels

5.3.1. Personen

Elke persoon:

- heeft een uniek **RijksregisterNr** (RRN).
 - een **RijksregisterNr** heeft een validering (koppeling met geboortedatum; controle-getallen) die je in code moet verifiëren.
 - bij de bronnenlijst vind je meer info over de validering van een rijksregisternummer.
- een patiënt kan niet tweemaal ingeschreven zijn op dezelfde afdeling.

5.3.2. Artsen

Elke **Arts**:

- heeft een uniek **RIZIV-nummer** (11 cijfers).
- heeft exact één specialiteit.
- is minstens verbonden aan één afdeling, mogelijks aan meerdere
 - een arts kan niet tweemaal verbonden zijn aan dezelfde afdeling.

5.3.3. Afspraken

Elke **Afspraak**:

- Heeft **exact één Patiënt**, één **Arts** en één **Afdeling**.
- Het moet gaan over een controle, een operatie of een consultatie.
- De patiënt moet ingeschreven zijn op de afdeling van de afspraak.
- De arts moet verbonden zijn aan de afdeling van de afspraak.
- Een **Controle**-afspraak verwijst verplicht naar **exact één** eerdere afspraak.
 - Het tijdstip van deze controle afspraak dient later te zijn dan de eerdere afspraak.
 - De controle afspraak dient te gebeuren op dezelfde afdeling als de eerdere afspraak en voor dezelfde patiënt.
- Een patiënt mag maar één afspraak hebben bij dezelfde arts op dezelfde dag.

- Een **Operatie** bevat een niet-lege **beschrijving** en **duur** ≥ 30 minuten.
- Een **Consultatie** heeft een **duur** ≥ 10 minuten.
- Een Afspraak kan niet overlappen voor **dezelfde arts** (geen twee afspraken die in tijd overlappen). Idem voor **dezelfde patiënt**.



Aangezien een arts ook een patiënt kan zijn stellen we een regel op: er mag geen afspraak geboekt worden waarbij de arts en patiënt dezelfde persoon zijn. Het zou vreemd ogen moest een arts zichzelf opereren!?

5.4. Unit test

Alle domeinregels dienen voldoende getest te worden.

6. Te bouwen applicaties



Voor deze opdracht worden **twee afzonderlijke applicaties** ontwikkeld: een **Gegevensinitialisatie-applicatie** (WPF) en een **Gebruikersapplicatie** (WPF). Beide applicaties worden gerealiseerd volgens een meerlagenarchitectuur bestaande uit de lagen **Presentatie**, **Application Service**, **Domein** en **Infrastructuur**. Voor de implementatie worden **WPF**, **ADO.NET** en **xUnit** gebruikt.

De twee applicaties maken gebruik van dezelfde Application Service-, Domein- en Infrastructuurlaag. Het ontwerp moet daarom maximaal inzetten op herbruikbaarheid, zodat gemeenschappelijke functionaliteit slechts één keer wordt geïmplementeerd en door beide applicaties kan worden gebruikt.

7. Gegevensinitialisatie-applicatie

Ontwikkel een **grafische applicatie** waarmee de aangeleverde tekstbestanden kunnen worden ingelezen en de gegevens in een databank worden opgeslagen. Besteed hierbij eerst aandacht aan het ontwerp van het domein en de Application Services voordat de technische verwerking wordt uitgewerkt.

Aanbevolen aanpak:

- Werk eerst het **domeinmodel** en de **Application Services** uit.
- Implementeer vervolgens het inlezen en verwerken van de bestanden. Als tussenstap mogen de ingelezen gegevens tijdelijk naar de console worden gelogd.
- Zodra de verwerking correct functioneert, ontwerp je de databankstructuur en stel je een **ERD (Entity Relationship Diagram)** op.



Bij elke gegevensinitialisatie moet de databank eerst worden leeggemaakt voordat nieuwe gegevens worden geladen. Op die manier start elke verwerking vanuit een

consistente en propere toestand.



Wanneer het domein gebruikmaakt van overerving (bijvoorbeeld bij afspraken), moet de databank een afzonderlijke tabel bevatten voor de eigenschappen van de superklasse en bijkomende tabellen voor de eigenschappen van de respectieve subklassen.

Gebruik van de applicatie:

1. Laat de gebruiker de bestanden **Artsen.csv**, **Patienten.csv** en **Afspraken.csv** selecteren.
 - a. Wanneer een ongeldig bestand wordt geselecteerd, krijgt de gebruiker een duidelijke foutmelding.
 - b. Een bestand wordt als ongeldig beschouwd wanneer:
 - i. de extensie niet **.csv** is; of
 - ii. de eerste regel van het CSV-bestand niet overeenkomt met de verwachte header.
2. Bouw de databank op of herbouw ze wanneer deze reeds bestaat (bij herladen worden de relevante tabellen eerst geleegd).
3. Lees de geselecteerde bestanden in en verwerk de gegevens.
4. Rapporteer na afloop het aantal succesvol geïntialiseerde **Artsen**, **Patiënten** en **Afspraken**.
5. Voorzie een uitgebreide foutafhandeling.
 - a. Meld expliciet wanneer één of meerdere foutbestanden werden aangemaakt.
 - b. Werk robuust: fouten in een individuele lijn mogen de verwerking van de overige lijnen niet verhinderen.
6. Gebruik **transacties** wanneer gegevens tijdens één bewerking in meerdere tabellen moeten worden opgeslagen (bijvoorbeeld bij het bewaren van een afspraak samen met de bijbehorende subtypegegevens).



Denk vanaf het begin na over herbruikbaarheid. Functionaliteit die in deze applicatie wordt ontwikkeld, moet zonder copy-paste kunnen worden hergebruikt in de gebruikersapplicatie.



Tijdens de mondelinge verdediging kunnen door de lector nieuwe tekstbestanden worden aangeleverd. De applicatie moet daarom voldoende generiek en robuust zijn om ook deze bestanden correct te kunnen verwerken.

8. Gebruikersapplicatie Zorgcentrum (WPF)

Na het uitvoeren van de gegevensinitialisatie beschikt deze gebruikersapplicatie over een relationele databank met minstens artsen, patiënten en afspraken. Nu bouw je een WPF-gebruikersapplicatie waarmee eindgebruikers deze gegevens op een gecontroleerde en gebruiksvriendelijke manier kunnen raadplegen en beheren.



Ga waar mogelijk code hergebruiken.

De applicatie wordt opgebouwd volgens een meerlagenarchitectuur (Presentatie – Applicatie service - Domein – Infrastructuur) en gebruikt ADO.NET voor data-toegang. Domeinregels worden uitsluitend afgedwongen in de domeinlaag en getest via xUnit. De UI mag geen businesslogica bevatten.

8.1. Algemeen concept: keuze gebruikersmodus

Bij het opstarten van de applicatie krijgt de gebruiker eerst een welkomscherm te zien. Dit scherm heeft als doel om de gebruikersmodus te bepalen waarin de applicatie zal werken.

De gebruiker kan kiezen tussen:

- Read-only modus
- Admin modus

Deze keuze bepaalt welke functionaliteit in de rest van de applicatie beschikbaar is. In read-only modus kan de gebruiker gegevens raadplegen maar geen enkele wijziging uitvoeren. In admin modus zijn alle beheersfuncties beschikbaar.

8.2. Scherm 1 – Welkom & modusselectie

Het eerste scherm is een eenvoudig, duidelijk welkomscherm met een korte toelichting over het doel van de applicatie. De gebruiker kiest hier expliciet voor Read-only of Admin via twee knoppen.

Na deze keuze wordt de gebruiker doorgestuurd naar het hoofdscherm. De gekozen modus blijft actief gedurende de volledige sessie en beïnvloedt:

- zichtbaarheid van knoppen
- beschikbaarheid van acties

8.3. Scherm 2 – Overzicht van afspraken (dashboard)

Het hoofdscherm van de applicatie is een overzichtsscherm voor afspraken, dat fungeert als dashboard. Dit scherm is beschikbaar in beide modi, maar gedraagt zich anders afhankelijk van de gekozen gebruikersrol.

De gebruiker ziet hier een overzicht van afspraken.



Denk voor je start eens goed na over de user interface. Ga je standaard alle afspraken die bestaan tonen of verplicht je de gebruiker om eerst te filteren op bv minstens één afdeling voor je afspraken toont. Vooral: zorg dat de applicatie werkbaar is voor de gebruiker maar let ook op de communicatie met de database en de hoeveelheid data die je uitwisselt.

Het scherm laat toe om afspraken te zoeken en te filteren. Het is aan jou om de mogelijkheden van

filteren te bepalen! Je zou kunnen:

- filteren op afdeling
- filteren door één of meer artsen uit een lijst te kiezen
 - de lijst van artsen zou je kunnen opzoeken door te filteren op naam.
- filteren door een patiënt uit een lijst te kiezen
 - een patiënt zou je kunnen opzoeken door te filteren op naam
- afspraken filteren van een patiënt bij een specifieke arts op bv.:
 - afspraken vandaag
 - in het verleden
 - in de toekomst of
 - binnen een zelfgekozen periode: bv. standaard worden de toekomstige afspraken getoond voor de volgende vijf dagen, maar de gebruiker kan dit aanpassen.

In read-only modus kan de gebruiker dit scherm volledig gebruiken om informatie te raadplegen, maar zijn acties zoals toevoegen, bewerken of verwijderen niet beschikbaar. In admin modus verschijnen extra acties zoals:

- nieuwe afspraak aanmaken
- bestaande afspraak openen om te bewerken
- één of meerdere afspraken verwijderen
- een nieuwe patiënt aanmaken of een bestaande patiënt inschrijven in een afdeling



Er zijn ook domeinregels die roet in het eten kunnen gooien: overlappende afspraken voor dezelfde arts of patiënt mogen bijvoorbeeld niet voorkomen. Indien ingegeven data problematisch is, wordt dit visueel aangeduid op het moment dat men de data probeert op te slaan.

8.4. Scherm 3 – Afspraakdetail (bekijken en beheren)

Wanneer de gebruiker een afspraak selecteert in het overzicht, wordt een detailweergave geopend. Dit is een apart scherm.

In dit scherm worden alle gegevens van de afspraak getoond: type, datum en tijdstip, duur, patiënt, arts, afdeling en eventuele extra informatie. Bij een controle-afspraak wordt ook de info van geselecteerde oorspronkelijke afspraak waarop gecontroleerd wordt kort en duidelijk weergegeven.

De UI moet gebruikers begeleiden met duidelijke foutmeldingen wanneer domeinregels geschonden worden, zoals overlappende afspraken, ongeldige duur of een controle die verwijst naar een latere afspraak.

8.5. Scherm 4 – Beheer van patiënten

In admin modus beschikt de applicatie over een extra scherm waarin patiënten beheerd kunnen worden. Dit scherm is niet toegankelijk in read-only modus.

De gebruiker kan hier:

- nieuwe patiënten toevoegen
- bestaande gegevens aanpassen
- gekoppelde afdelingen raadplegen en wijzigen



Een scherm om Artsen toe te voegen of te wijzigen wordt niet gevraagd.

8.6. Exportfunctionaliteit

Vanuit het overzichtsscherm moet de gebruiker afspraken kunnen exporteren naar een tekstbestand in AsciiDoctor-formaat. De export is mogelijk per:

- geselecteerde afspraken
- arts
- patiënt
- periode

De gegenereerde export bevat een overzichtelijke structuur, geschikt om later om te zetten naar PDF of HTML.

9. Architectuur en puntenverdeling

Je maakt gebruik van een architectuur met meerdere lagen:

1. Presentatie
2. Applicatie service
3. Domein
4. Infrastructuur

9.1. Applicatie service- en Domeinlaag (40%)



Leid uit de opgave de domeinregels af. Alle domeinregels moeten worden afgedwongen in de domeinlaag en niet in de presentatie-, applicatie service- of infrastructuurlaag. Ontwikkel een domeinmodel met klassen en gebruik geen DataTable op je domeinlaag.

- Let op de relaties tussen de klassen: maak voor jezelf eventueel een klassendiagram.



Maak waar nodig gebruik van value objects, entiteiten, aggregates, domein services, applicatie services en DTO objecten.

9.2. Unit testen (5%)

- Voorzie unit testen voor de domeinregels van toepassing op je modellen, en dit in een afzonderlijk project en voor je models uit het domein. Gebruik het xUnit framework.



Belangrijk is minstens de nodige testen uit te voeren voor de domeinregels.

9.3. Infrastructuurlaag (15%)

- Primary key id's die aangemaakt dienen te worden voor je entiteiten ga je ofwel zelf aanmaken (GUID genereren) ofwel laat je dit gebeuren door de databank (maak gebruik van IDENTITY(1,1)).



Gebruik transacties waar nodig; dit is zeker van belang wanneer je bij een bepaalde operatie in meer tabellen tegelijk moet bewerken en alle schrijfacties succesvol moeten uitgevoerd zijn alvorens de operatie als 'gelukt' beschouwd mag worden.

- Let erop dat je parameters gebruikt waar dit kan (om SQL Injection te voorkomen).

9.4. Gegevensinitialisatie app (10%)

- Maak een afzonderlijke grafische applicatie om gegevens uit tekstbestanden op te laden naar je databanktabellen (= CVS bestanden parsen en hiermee de databank opvullen)
- Maak de tabellen van je databank eerst leeg alvorens je het opladen start.

9.5. Presentatie laag (30 %)

- Maak de presentatie laag met WPF. Denk eerst na over een wireframe voor je schermen/functionaliiteit. Denk na over je schermen zelf en de daarop aanwezige functionaliteit en over de manier van navigeren tussen de schermen.



Zorg ervoor dat je vanaf elk scherm kan terugkeren naar het vorige scherm.



Volg het drielagen model: alle informatie die je op het scherm toont wordt aan de domeinlaag opgevraagd!

10. Legendes

Table 1. Specialiteiten

Code	Specialiteit
ANES	Anesthesie-reanimatie
BIOL	Klinische biologie
CARD	Cardiologie
CHIR	Heelkunde
DERM	Dermatologie
ENDO	Endocrino-diabetologie
GAST	Gastro-enterologie
GERC	Geriatric
GYNA	Gynaecologie & verloskunde
HEMA	Klinische hematologie
NCHI	Neurochirurgie
NEFR	Nefrologie
NEUR	Neurologie
NUCL	Nucleaire geneeskunde
ONCO	Medische oncologie
ORL	Oto-Rhino-Laryngologie
OPHT	Oftalmologie
ORTH	Orthopedische heelkunde
PAAN	Pathologische anatomie
PEDI	Kindergeneeskunde
PNEU	Pneumologie
PSYC	Psychiatrie
RADO	Radiotherapie-oncologie
REUM	Reumatologie
REVA	Fysische geneeskunde & revalidatie
RXDI	Röntgendiagnose
SLAP	Slaapgeneeskunde
STOM	Stomatologie
URGE	Urgentiegeneeskunde
UROL	Urologie

Table 2. Afdelingen

Code	Afdeling
PNEC	Pneumologie – Slaapcentrum

Code	Afdeling
CARD	Cardiologie-afdeling
GAST	Gastro-enterologie afdeling
NEUR	Neurologie afdeling
ORTH	Orthopedie afdeling
GERC	Geriatric afdeling
ONCO	Oncologie afdeling
PAAN	Pathologische anatomie
RADI	Radiologie / MRI / CT
URGE	Spoed & urgentiegeneeskunde

11. Te gebruiken technologieën

- Visual Studio Enterprise 2026
- C#
- .NET 10.0
- SQL server Express 2025 en ADO.NET
- GIT
- WPF
- xUnit
- Excepties om echte uitzonderlijke gebeurtenissen op te vangen
- Architectuur uit meerdere lagen: Presentatie - Application Service - Domein - Infrastructuur

12. Bronnen (validatieregels)

- **Rijksregisternummer (RRN)** – controlegetal en 2000-regel (97 – (B mod 97)) en structuur YYMMDD-XXX-CC:
 - Vlaamse overheid – MAGDA validatiemethodes (confluence): [vlaamseoverheid.atlassian.net/wiki/spaces/MG/pages/6208324471/Validatiemethodes](https://confluence.vlaamseoverheid.atlassian.net/wiki/spaces/MG/pages/6208324471/Validatiemethodes)
 - FOD Binnenlandse Zaken – Rijksregister onderrichting IT000: www.ibz.rrn.fgov.be/sites/default/files/documents/nl/rijksregister/onderrichtingen/IT-lijst/IT000_Rijksregisternummer.pdf